

Alocação de Computação de Inferência na Síntese de Programas com Modelos de Linguagem

Anonymous submission

Resumo

Um orçamento fixo de chamadas a um modelo de linguagem admite duas alocações incompatíveis: *diversificar*, gerando N candidatos independentes, ou *iterar*, revisando um único candidato com feedback. A literatura relata ganhos para a segunda via, mas também falhas sistemáticas de autocorreção — e as duas leituras são difíceis de separar porque, sem informação externa, o revisor não sabe que errou. Removemos esse confundimento por construção, concedendo à via iterativa uma vantagem que ela não teria em produção: um Agente Crítico que enxerga o gabarito do par de teste e devolve apenas contradições em linguagem natural, sem propor solução e sem participar da seleção do candidato final. Ele opera, assim, como um limite superior otimista da via iterativa. Em 270 tarefas do ARC-AGI-1, com quatro condições sob orçamento igualado, **nenhuma** das 5 comparações pré-registradas é significativa — nem sob Bonferroni, nem a $\alpha = 0,05$ sem correção. O achado mais transferível independe disso: cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas. Descontada essa parcela, a medição discrimina em cerca de um quinto da amostra e a ordenação entre as condições inverte em relação ao número bruto — um confundimento herdado por qualquer comparação entre esquemas de inferência que partam do mesmo *prompt*, e que nenhuma métrica agregada revela.

1 Introdução

Uma fração substancial do desempenho recente em tarefas de raciocínio vem de escalar a computação no momento da inferência, e não o modelo (Snell et al. 2024; Brown et al. 2024). Isso transforma uma questão antes implícita em decisão de engenharia com consequência direta de custo: dado um orçamento fixo de chamadas, como alocá-lo?

Há duas respostas estruturalmente distintas. **Diversificar** é gerar N programas independentes e selecionar o melhor por um verificador barato — a receita de *pass@k* em geração de código (Chen et al. 2021), da filtragem massiva do AlphaCode (Li et al. 2022) e da autoconsistência em raciocínio (Wang et al. 2023). **Iterar** é gastar o mesmo orçamento revisando um único candidato à luz de feedback, como em

self-debugging (Chen et al. 2024), SELF-REFINE (Madaan et al. 2023) e REFLEXION (Shinn et al. 2023).

A evidência sobre a segunda via é contraditória. Os trabalhos originais relatam ganhos expressivos; réplicas controladas encontram que grande parte deles desaparece quando o orçamento é igualado, ou quando o modelo precisa detectar sozinho que errou (Huang et al. 2024; Stechly, Marquez, and Kambhampati 2023; Olausson et al. 2024). A explicação candidata mais simples é que autocrítica não é crítica: um modelo sem acesso a informação externa sobre o próprio erro tende a reafirmar a resposta que já produziu. Sob essa hipótese, o fator limitante não é a estratégia iterativa, e sim a qualidade do canal de feedback.

Testamos essa hipótese maximizando exatamente a variável que ela aponta. Nosso Agente Crítico enxerga o gabarito do par de teste, que o Gerador nunca vê, e sua função é restrita por construção: não propõe solução, não escreve código e não reproduz grades, limitando-se a apontar contradições entre a regra enunciada, o programa candidato e a realidade. O desenho é, portanto, deliberadamente favorável à via iterativa — o crítico dispõe de informação privilegiada que nenhum sistema autônomo teria e ainda assim consome o mesmo orçamento de chamadas. Ele funciona como um **limite superior otimista**, o que torna a leitura do resultado informativa nos dois sentidos: se o crítico vencer, o gargalo da autocorreção é informacional; se não vencer, a vantagem atribuída ao feedback iterativo fica sob suspeita muito além deste arranjo.

Avaliamos no ARC-AGI-1 (Chollet 2019), em que cada tarefa traz alguns pares entrada → saída de demonstração e exige inferir a transformação subjacente. O domínio é adequado por três razões: a resposta é verificável exatamente, o que elimina a ambiguidade de métricas parciais; os pares de demonstração fornecem um verificador barato e honesto que ambas as estratégias podem usar sem tocar no gabarito; e o *benchmark* resiste a modelos de propósito geral (Gendron et al. 2024), deixando espaço de medição.

Contribuímos com um protocolo pareado de orçamento igualado que isola a decisão diversificar vs. iterar, com salvaguardas auditáveis contra contaminação pelo oráculo; com uma decomposição fatorial do mecanismo em dois eixos — acesso ao oráculo e forma do feedback — em 270 tarefas; e com a quantificação de quanto de uma comparação entre arquiteturas de agentes é de fato atribuível à arquitetura.

2 Trabalho Relacionado

Amostrar muitos candidatos e filtrar é a estratégia de referência em síntese de programas com modelos de linguagem (Chen et al. 2021; Li et al. 2022), e trabalhos recentes caracterizam a curva: a cobertura cresce de forma previsível com o número de amostras (Brown et al. 2024), e há regimes em que alocar computação de inferência supera aumentar o modelo (Snell et al. 2024). Nossa condição `sampling` é uma instância deliberadamente simples dessa família, com o verificador mais honesto que o ARC oferece sem custo — os próprios pares de demonstração —, o que mantém a comparação livre de qualquer componente treinado.

A via iterativa realimenta o modelo com o resultado da execução ou com uma crítica textual e solicita revisão (Chen et al. 2024; Madaan et al. 2023; Shinn et al. 2023). A contestação identifica a qualidade do sinal externo como fator determinante: Huang et al. (2024) argumentam que modelos não corrigem raciocínio sem sinal externo, Stechly, Marquez, and Kambhampati (2023) mostram que a melhora aparente provém do verificador embutido no protocolo, e Olausson et al. (2024) constataam que o auto-reparo só compensa quando o crítico é mais capaz que o gerador. Nosso desenho toma essa variável e a maximiza. Guiar síntese por contraexemplos, por sua vez, antecede os modelos de linguagem: em CEGIS, um verificador devolve ao sintetizador uma entrada concreta em que o candidato falha (Solar-Lezama et al. 2006). Nossa condição `critic_cegis` transpõe esse protocolo para o registro de linguagem natural, o que permite variar a forma do feedback com o conteúdo informacional constante.

3 Metodologia

3.1 Formulação e Condições

Uma tarefa do ARC-AGI-1 traz pares de demonstração $\mathcal{D}_t$ e um par de teste (x_t^*, y_t^*) . Um solucionador recebe $\mathcal{D}_t$ e x_t^* e produz um programa π ; a tarefa é resolvida se e somente se $\pi(x_t^*) = y_t^*$ com igualdade exata da grade. O recurso controlado é o **orçamento de chamadas** B : o número máximo de requisições ao modelo por tarefa, somando todos os papéis de agente. A igualdade de B entre as condições é o controle central — sem ela, a condição com mais recursos venceria por dispor de mais recursos.

Em todas as condições, o Gerador propõe uma regra em linguagem natural e um programa, que é executado em *sandbox* sobre os pares de demonstração. Se todos forem reproduzidos, o laço termina; caso contrário, e havendo orçamento, o Crítico examina o candidato e devolve feedback ao Gerador. A Tabela 1 define as quatro condições comparadas. `sampling` aloca o orçamento em largura: cada chamada é uma conversa nova com o mesmo *prompt*, e ao final vence o candidato que reproduz mais pares de demonstração. As três condições de crítico alocam em profundidade, gastando duas chamadas por ciclo, e variam um eixo cada em relação a `critic`: `critic_no_oracle` nunca vê y_t^* , isolando quanto do efeito vem da estrutura da crítica sem informação privilegiada; `critic_cegis` vê y_t^* mas responde com um contraexemplo e uma classe de correção em vocabulário fechado, isolando o efeito da forma do feedback.

	samp	critic	no_or	cegis
Feedback	—	prosa	prosa	estrut.
Vê y_t^*	não	sim	não	sim
Histórico (Gerador)	não	sim	sim	sim
Temperatura	0,8	0,2	0,2	0,2
Chamadas por ciclo	1	2	2	2
Programas/tarefa	7	4	4	4
Desempate	1 ^a	últ.	últ.	últ.

Tabela 1: As quatro condições (`no_or` = `critic_no_oracle`, `cegis` = `critic_cegis`), todas sob o mesmo orçamento B de chamadas. A temperatura é assimétrica por necessidade: a $T = 0,2$ as N amostras colapsam em programas quase idênticos e o *best-of- N* degenera em uma tentativa repetida. “Desempate” indica qual candidato vence quando há empate na consistência com os pares de demonstração — a última iteração onde há progressão, a primeira amostra onde não há.

3.2 Salvaguardas Contra Contaminação pelo Oráculo

Conceder o gabarito a um agente cria três riscos, tratados por mecanismos distintos. **O gabarito não seleciona:** o critério de parada e a escolha do candidato final olham apenas para os pares de demonstração, nas quatro condições, e o par de teste é executado uma única vez ao final, apenas para medir. Se o laço parasse ao acertar o teste, a taxa de acerto seria um limite superior de oráculo, não uma medida de desempenho. **O gabarito não vaza:** o feedback passa por um filtro que redige linhas compostas apenas de inteiros e qualquer código antes de chegar ao Gerador, com a resposta bruta preservada para auditoria e cada redação contabilizada — o vazamento deixa de depender da cooperação do modelo e torna-se mensurável. **O Crítico não resolve:** ele é reinstanciado a cada rodada, sem histórico, de modo que não pode construir uma solução ao longo do diálogo e virar um segundo solucionador.

3.3 Procedimento e Análise

As quatro condições rodam sobre as mesmas 270 tarefas sorteadas de `evaluation` com semente 20260814 e $B = 7$ (ímpar, para que a última chamada seja sempre do Gerador), com `gemin-3.5-flash-lite` nos dois papéis — usar o mesmo modelo isola a assimetria de informação da capacidade de um segundo modelo. A escolha do modelo é uma escolha de poder estatístico: modelos fortes resolvem quase tudo já na primeira geração e modelos fracos falham em tudo, e ambos os extremos produzem apenas pares concordantes; `gemin-3.5-flash-lite` maximiza a taxa de discordância entre os modelos acessíveis.

O desenho é pareado, então o que importa não é a diferença de acurácia agregada, mas quantas tarefas cada condição resolveu que a outra não. Usamos o teste de McNemar exato (McNemar 1947) sobre os pares discordantes, preferido à aproximação qui-quadrado por ser confiável com poucas discordâncias, e o intervalo de confiança de Wilson (Wilson 1927) sobre a proporção de discordâncias, convertido para pontos percentuais de acurácia. Reportar o intervalo é

Condição	Res.	Acur.	Treino	Prog.	Tok.
critic	93/270	34,4%	35,6%	3,23	48,1 k
sampling	88/270	32,6%	35,6%	5,43	28,0 k
cegis	83/270	30,7%	34,8%	3,34	48,8 k
no_oracle	80/270	29,6%	33,3%	3,27	45,8 k

Tabela 2: Resultados agregados, ordenados por acurácia. “Treino” é a fração de tarefas em que o candidato final re-produz todos os pares de demonstração — o alvo que as estratégias de fato otimizam.

Par	Discord.	p	IC 95%
samp-critic	51 (23×28)	0,5758	−3,3 a +6,7
samp-no_or	54 (31×23)	0,3409	−7,9 a +2,3
samp-cegis	47 (26×21)	0,5601	−6,5 a +3,0
critic-no_or	47 (30×17)	0,0789	−9,1 a +0,2
critic-cegis	54 (32×22)	0,2203	−8,5 a +1,6

Tabela 3: As 5 comparações pré-registradas, com p de McNemar exato e intervalos em pontos percentuais. O menor p observado é 0,0789, e todos os intervalos incluem o zero.

o que distingue a ausência de efeito detectado da ausência de capacidade de medição. A família de 5 comparações foi pré-registrada antes da execução, o que exige a correção de Bonferroni (Dunn 1961): $\alpha = 0,05/5 = 0,01$ por teste. Tarefas interrompidas por erro de API são excluídas da comparação pareada.

4 Resultados

A medição (02–03/09/2026) emitiu 1.844 críticas e registrou 0 eventos de vazamento. As Tabelas 2 e 3 trazem os agregados e as comparações pareadas.

Sob orçamento igualado, **nenhuma** das 5 comparações pré-registradas é significativa — nem sob Bonferroni, nem a $\alpha = 0,05$ sem correção. A comparação mais próxima é *critic* vs. *critic_no_oracle*, com $p = 0,0789$; a diferença entre *critic* e *sampling* é de +1,9 pp, com intervalo de −3,3 a +6,7 pp.

O mecanismo, porém, opera como projetado: o Crítico produz 3,23 programas por tarefa contra 5,43 da amostragem — 40% a menos — atingindo a *mesma* consistência com os pares de demonstração (35,6% nas duas condições). Converte orçamento em revisão com eficiência real; o que não ocorre é a conversão disso em acurácia no teste. A restrição de formato de *critic_cegis* também foi obedecida sem exceção (630/630 das críticas com rótulo válido de classe de correção), sem que a adesão se traduzisse em vantagem. E *critic_no_oracle*, a única condição que descreve um sistema executável sem gabarito, é a de menor acurácia bruta (29,6%): nesta escala, crítica estruturada sem informação privilegiada não compra nada mensurável.

4.1 A Origem das Vitórias

A primeira chamada é idêntica nas quatro condições — mesmo *prompt*, histórico vazio, nenhum crítico envolvido —, diferindo apenas na temperatura. Vitórias decididas ali

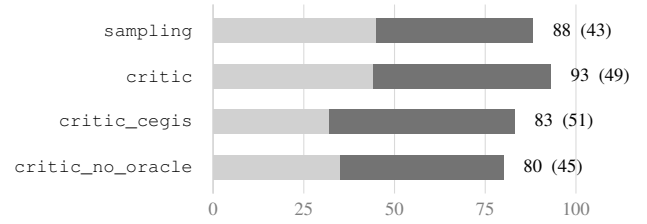


Figura 1: Decomposição das vitórias, em tarefas resolvidas de 270. O segmento claro são as decididas na primeira geração, comum às quatro condições; o escuro, as atribuíveis ao ciclo. Os rótulos dão o total e, entre parênteses, a parcela do ciclo. A ordenação pelo total (93, 88, 83, 80) *inverte* quando se olha apenas o ciclo (51, 49, 45, 43): as três condições de crítico passam à frente da linha de base.

não distinguem estratégia alguma. A Figura 1 separa, em cada condição, as vitórias obtidas na primeira geração das obtidas pelo ciclo.

O resultado é que cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas. Restringindo cada par às tarefas que a primeira geração não resolveu em nenhuma das duas condições, restam 207–216 tarefas — cerca de um quinto da amostra discrimina de fato — e todos os 5 p -valores ficam entre 0,2962 e 1,0000. O mesmo padrão aparece numa medição anterior sobre as mesmas tarefas, com duas condições e um protocolo de crítico diferente: descontada a primeira geração, o placar fica empatado em 44×44 sobre 206 tarefas.

Dois leituras decorrem daí. A primeira é metodológica: o poder estatístico efetivo de uma comparação entre arquiteturas de agentes é muito menor do que n sugere. A segunda vai na direção oposta ao número bruto: descontada a primeira geração, as três condições de crítico ficam *acima* de *sampling* em vitórias do ciclo (51, 49 e 45 contra 43). As diferenças são pequenas e nenhuma é significativa, mas a direção depende de qual quantidade se observa — dependência invisível em qualquer relato que reporte apenas acurácia agregada. O confundimento é eliminável por construção: basta que a primeira condição a alcançar uma tarefa grave a primeira geração e as demais a repitam, com a repetição ainda debitada do orçamento, o que de quebra economiza 810 chamadas em um bloco de quatro condições.

5 Discussão

A leitura defensável é pragmática: sob orçamento de chamadas igualado, alocar o orçamento em diversidade não é distinguível de alocá-lo em revisão dirigida — nem mesmo quando a revisão é guiada por um oráculo indisponível a qualquer sistema autônomo. Como o desenho favorece a via iterativa por construção, o resultado é mais informativo do que um empate entre esquemas equivalentes: indica que a informação privilegiada, isoladamente, não é o fator limitante. Não sustenta, porém, a leitura causal de que a assimetria de informação seja irrelevante — as condições diferem em vários eixos, e os intervalos de confiança admitem efeitos que a medição não teria poder para detectar.

O achado mais consequente é o outro, e ele não é específico deste arranjo: qualquer comparação entre esquemas de inferência que partam do mesmo *prompt* inicial herda o confundimento da primeira geração, e nenhuma métrica agregada o revela. Uma verificação por subamostragem sem reposição das próprias 270 tarefas (20.000 repetições) mostra a consequência prática: com 30 tarefas, a direção observada em 270 se reproduz em apenas 51% das subamostras — indistinguível de um sorteio — e a ordenação completa das quatro condições, em 8%; com 60 tarefas, em 12%. Estudos desta classe conduzidos em algumas dezenas de tarefas não estão medindo direção (Button et al. 2013).

Limitações. A dominante é o confundimento da primeira geração, que reduz o n efetivo e cuja correção, embora conhecida e barata, não foi aplicada aqui. Seguem-se: o orçamento é medido em chamadas, não em tokens — as condições de crítico consomem de 63% a 74% mais tokens por tarefa, e sob orçamento em tokens o sinal poderia inverter; os oráculos não são autônomos, de modo que `critic` e `critic_cegis` medem a validação por oposição como limite superior, não um método implantável; há uma execução por tarefa, então parte da variação é ruído do modelo, mitigado pelo pareamento mas não eliminado; `sampling` foi coletado em data anterior à das condições de crítico, e o modelo é hospedado, o que introduz um confundimento temporal nas comparações que envolvem a linha de base; e o filtro anti-vazamento captura grades e código, mas não paráfrases, de modo que ele mede o que foi barrado, não o que passou.

6 Conclusão

Sob orçamento fixo de chamadas para síntese de programas no ARC-AGI-1, quatro condições que variam isoladamente o acesso ao oráculo e a forma do feedback não se separam em 270 tarefas: **nenhuma** das 5 comparações pré-registradas é significativa — nem sob Bonferroni, nem a $\alpha = 0,05$ sem correção. O crítico opera como mecanismo, atingindo a mesma consistência com os pares de demonstração com 40% menos programas, sem que isso se converta em acerto no teste.

O resultado mais transferível independe de qual condição venha a prevalecer: cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas. Descontada essa parcela, a medição discrimina em cerca de um quinto da amostra e a ordenação entre as condições inverte. O passo seguinte não é aumentar n , e sim eliminar esse confundimento por construção e medir o orçamento em tokens, que é a unidade de custo efetiva.

Reprodutibilidade

Cada rodada grava um manifesto com a lista literal de identificadores de tarefa, a semente, o orçamento e os modelos, além de um registro por condição com todos os *prompts*, respostas brutas, códigos gerados e resultados de execução. As respostas brutas do Crítico são preservadas antes do filtro anti-vazamento, o que permite auditar independentemente o que foi redigido e o que passou. Os testes automatizados travam as propriedades críticas do desenho, incluindo a

separação de informação entre os *prompts* do Gerador e do Crítico e a paridade de orçamento entre as condições.

Referências

- Brown, B.; Juravsky, J.; Ehrlich, R.; Clark, R.; Le, Q. V.; Ré, C.; and Mirhoseini, A. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.
- Button, K. S.; Ioannidis, J. P. A.; Mokrysz, C.; Nosek, B. A.; Flint, J.; Robinson, E. S. J.; and Munafò, M. R. 2013. Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience. *Nature Reviews Neuroscience*, 14(5): 365–376.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Chollet, F. 2019. On the Measure of Intelligence. arXiv:1911.01547.
- Dunn, O. J. 1961. Multiple Comparisons Among Means. *Journal of the American Statistical Association*, 56(293): 52–64.
- Gendron, G.; Bao, Q.; Witbrock, M.; and Dobbie, G. 2024. Large Language Models Are Not Strong Abstract Reasoners. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)*, 6270–6278.
- Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Dal Lago, A.; et al. 2022. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624): 1092–1097.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 46534–46594.
- McNemar, Q. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2): 153–157.
- Olausson, T. X.; Inala, J. P.; Tan, C.; Gao, J.; and Solar-Lezama, A. 2024. Is Self-Repair a Silver Bullet for Code Generation? In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 8634–8652.

Snell, C.; Lee, J.; Xu, K.; and Kumar, A. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. arXiv:2408.03314.

Solar-Lezama, A.; Tancau, L.; Bodik, R.; Seshia, S.; and Saraswat, V. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 404–415.

Stechly, K.; Marquez, M.; and Kambhampati, S. 2023. GPT-4 Doesn't Know It's Wrong: An Analysis of Iterative Prompting for Reasoning Problems. arXiv:2310.12397.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.